Team Sigma Gamma

Jonathan Goohs, Onur Orkut Genişel

Project Focus: Lowering the barrier to SUMO contribution

1. Ideation

SigmaKEE is used by ontologists, students, and domain experts to author axioms, browse ontology structures, run proofs, and interpret results. The current interface presents all of these capabilities through a traditional web-based menu system with minimal guidance. Our team brainstormed independently to explore how a redesigned experience might better support these core workflows. Based on stakeholder input, our redesign targets the VS Code plugin as the primary modality, replacing SUMOjEdit which is being phased out due to setup friction and maintenance concerns.

Member 1 - Onur Orkut Genişel

My sketch focuses on a guided workflow approach. Instead of presenting all features at once, the interface walks the user through a clear sequence:

(1) connect to a knowledge base,

(2) browse or search for terms,

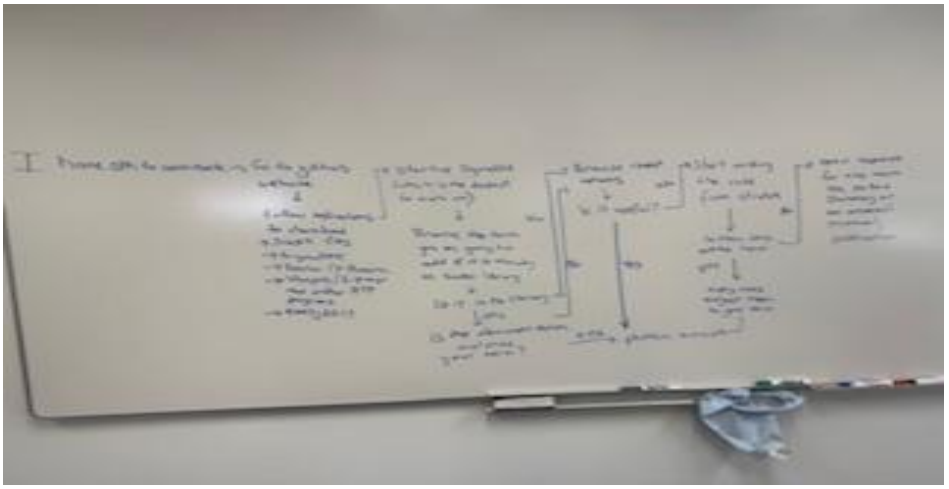
(3) author or edit an axiom,

(4) run a consistency check or proof, and

(5) review results. Each step has its own dedicated panel with contextual help.

This approach addresses the biggest pain points I experienced as a SigmaKEE user: not knowing where to start, losing context between editing and proving, and struggling to interpret proof output without guidance.

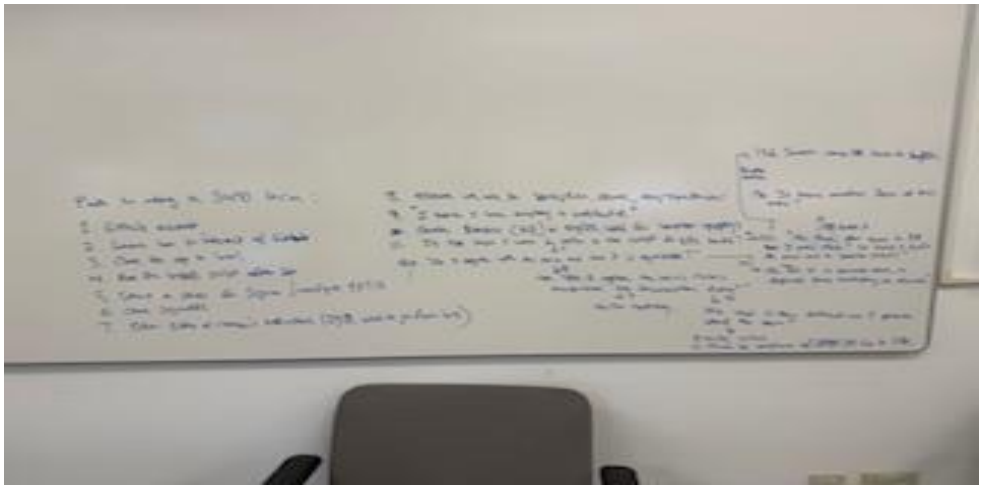
A secondary concern is setup friction. Configuring SUMOjEdit on macOS requires resolving path issues, symlinks, and Docker/XQuartz workarounds. Moving to a VS Code plugin eliminates this friction by leveraging an editor most students already use. The sketch includes a setup wizard as the entry point for first-time users, guiding them through environment configuration step by step.



Member 2 - Jonathan Goohs

Jonathan's sketch focuses on a three-phase guided flowchart for adding new terms to SUMO, designed as a VS Code extension wizard. The flowchart breaks the process into:

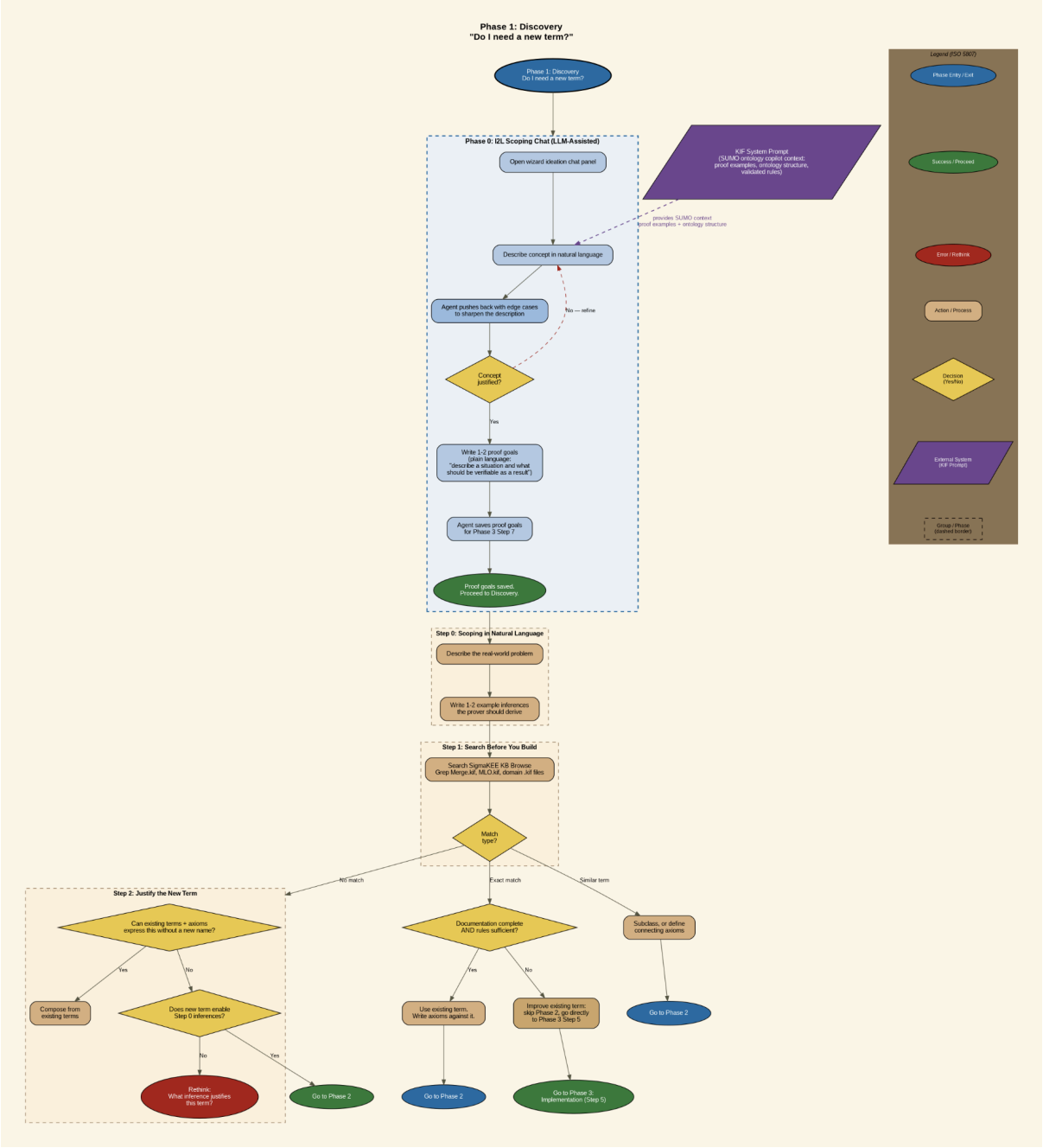
- Phase 1 - Discovery: Determines whether a new term is actually needed. The user describes the real-world problem in natural language, writes example inferences, then searches existing terms in SigmaKEE and .kif files. If an exact match exists, the user writes axioms against it. If a similar term exists, the user either defines a subclass or plans connecting axioms. If no match exists, the user must justify why a new term is necessary.
- Phase 2 - Classification: Determines what kind of term it is. Through a decision tree, the user classifies the term as an Attribute (Internal or Relational), Relation (Function or Predicate), Class, or Instance. Then the user identifies its parent in the SUMO hierarchy by determining argument count, Physical vs. Abstract distinction, and Object vs. Process distinction.
- Phase 3 - Implementation: Guides the user through writing the definition in SUO-KIF syntax, checking WordNet mappings, running diagnostics (KifFileChecker, SigmaKEE consistency check), and proving with Vampire. A final checklist verifies: documentation string present, parent class makes sense, domain/range for all relations, at least one inference-enabling axiom, no variable typos, prover returns Theorem, WordNet mapping in place, and KifFileChecker passes clean.



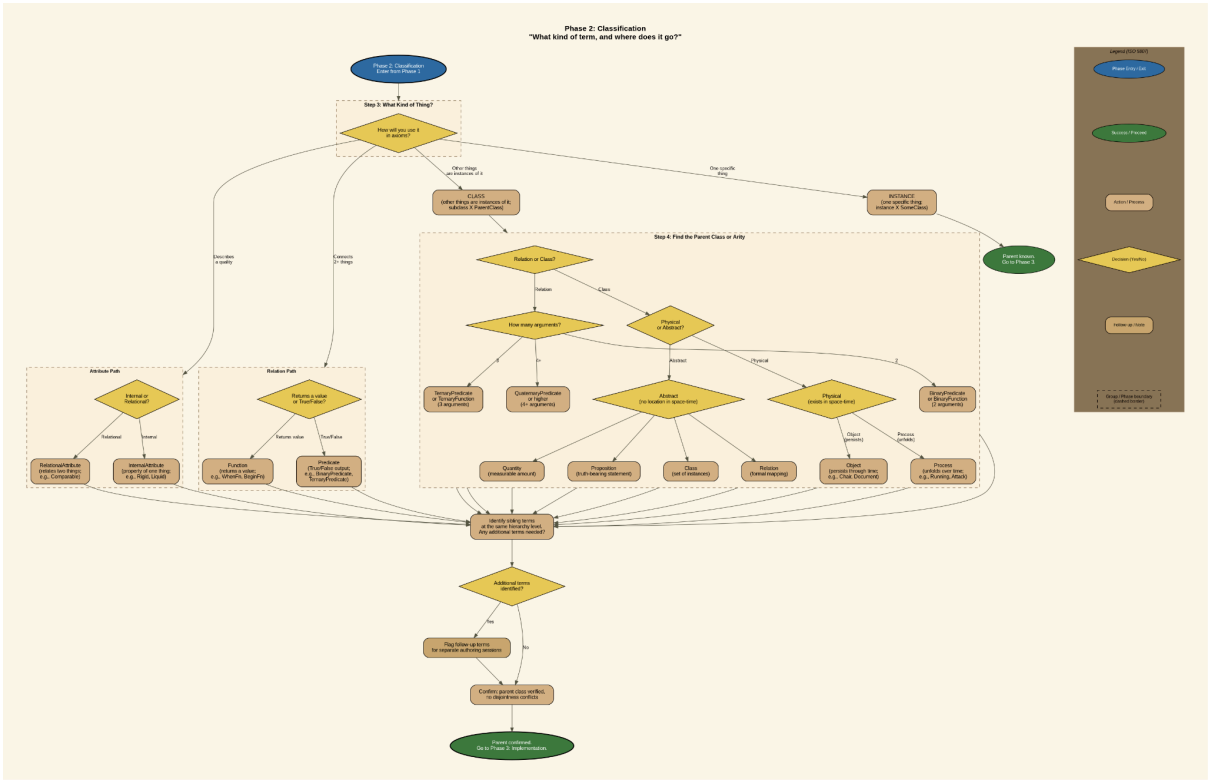
The flowchart sketches for all three phases are included below.

*****Note that this is a phasing flowchart codifying an optimal SUMO term about the world. This is not apart of the official documentation of Ontologyportal, and is the current basis for a draft academic work for submission by Jonathan Goohs.*****

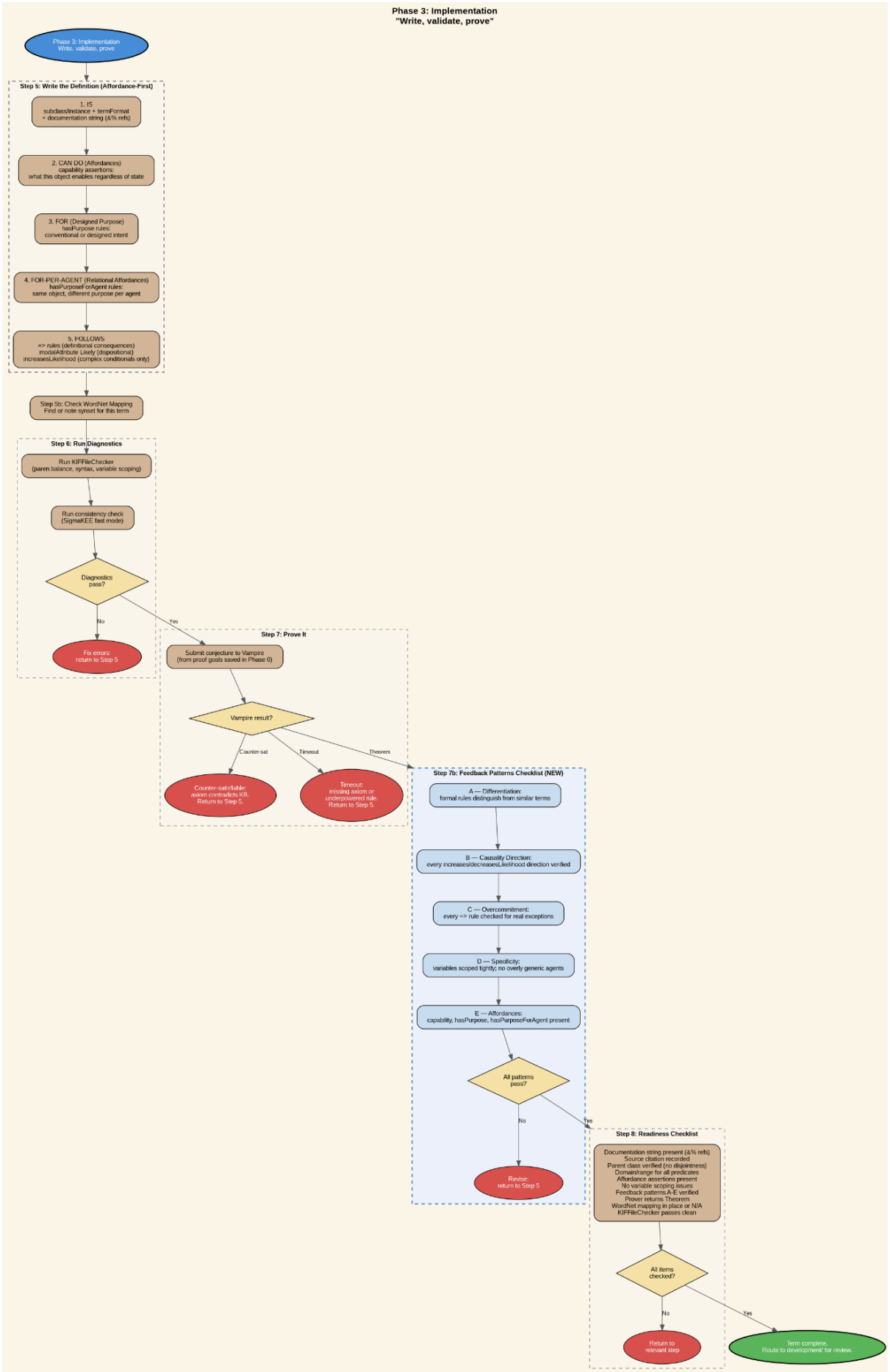
Phase 1: Discovery



Phase 2: Classification



Phase 3: Implementation


Phase 3: Implementation
"Write, validate, prove"
Phase 3: Implementation
Write, validate, prove
Step 5: Write the Definition (Affordance-First)
1. IS
subclass/instance + termFormat
+ documentation string (&% refs)
2. CAN DO (Affordances)
capability assertions:
what this object enables regardless of state
3. FOR (Designed Purpose)
hasPurpose rules:
conventional or designed intent
4. FOR-PER-AGENT (Relational Affordances)
hasPurposeForAgent rules:
same object, different purpose per agent
5. FOLLOWS
=> rules (definitional consequences)
modalAttribute Likely (dispositional)
increasesLikelihood (complex conditionals only)
Step 5b: Check WordNet Mapping
Find or note synset for this term
Step 6: Run Diagnostics
Run KIFFileChecker
(paren balance, syntax, variable scoping)
Run consistency check
(SigmaKEE fast mode)
Diagnostics
pass?
No
Fix errors:
return to Step 5
Yes
Step 7: Prove It
Submit conjecture to Vampire
(from proof goals saved in Phase 0)
Vampire result?
Counter-sat
Timeout
Theorem
Counter-satisfiable:
axiom contradicts KB.
Return to Step 5.
Timeout:
missing axiom or
underpowered rule.
Return to Step 5.
Step 7b: Feedback Patterns Checklist (NEW)
A — Differentiation:
formal rules distinguish from similar terms
B — Causality Direction:
every increases/decreasesLikelihood direction verified
C — Overcommitment:
every => rule checked for real exceptions
D — Specificity:
variables scoped tightly; no overly generic agents
E — Affordances:
capability, hasPurpose, hasPurposeForAgent present
All patterns
pass?
No
Revise:
return to Step 5
Yes
Step 8: Readiness Checklist
Documentation string present (&% refs)
Source citation recorded
Parent class verified (no disjointness)
Domain/range for all predicates
Affordance assertions present
No variable scoping issues
Feedback patterns A-E verified
Prover returns Theorem
WordNet mapping in place or N/A
KIFFileChecker passes clean
All items
checked?
No
Return to
relevant step
Yes
Term complete.
Route to development for review.

Combined "Best Of" Sketch

Our two sketches converge on the same core insight: the problem is not missing features, it is missing workflow guidance and efficiency. The combined design is a unified wizard embedded in VS Code that routes each user through a structured sequence of panels based on who they are and what they are trying to do.

The unified design layers Onur's five-panel interface sequence on top of Jonathan's three-phase decision flowchart. Onur's Setup Wizard panel becomes the onboarding entry point for new users. His Term Browser, Axiom Editor, and Proof Explorer panels correspond to the Discovery, Authoring, and Validation phases of Jonathan's flowchart. His Results and Review panel maps to the submission and routing step at the end. The same pipeline is delivered across three platform interfaces: CLI for engineers, VS Code for students, and a web or mobile app for domain experts, so each user arrives at the workflow through an environment they already know.

A key design decision from our ideation session: the wizard should not ask the user to choose between "add a new term" and "update an existing term." The system detects from search results which path applies and routes automatically. The user's only job at the start is to say: I have something I want to contribute.

Each user group enters the same pipeline but sees a different modality and happy path. The domain expert (User 3) gets the full guided experience, starting with a natural language scoping chat and moving through each phase with contextual prompts. The student (User 2) gets a simplified track with more explanations and fewer assumptions about prior knowledge. The power user (User 1) can skip the guided steps and drop directly into search and authoring. This can always be toggled as desired as users improve in skill level, desire more control, or want some higher ease of editing.

An ambitious future direction identified in our ideation is a multimodal input panel as the first step of the pipeline. The tagline would read: "Show us the way you see the world." The user could take a photo of the real-world thing they want to formalize, upload an image, or describe it in voice. The system would transcribe the input into a natural language description and feed it into the scoping chat. This makes the pipeline accessible to any domain expert, not just those who can type a precise concept definition. This represents the user-generated content input layer for open-source knowledge contribution.

Each phase transition in the wizard includes a confirmation checkpoint before advancing. Loading states during KB search, consistency check, and theorem proving display an animated Sumo mascot. Audio confirmation sounds play when an action completes successfully.

2. Data Collection

We used a survey as our primary data collection method. SigmaKEE has a small and specialized user base, so we distributed the survey directly to active users in the ontologyportal community. We collected 7 responses over a two-day collection window. The survey was designed to locate pain points in the actual authoring workflow and validate the guided wizard concept.

Interview Protocol

Survey Instrument

We used Google Forms to make the survey for its ease and automated data statistics. SurveyMonkey was capped for the free version at 10 questions.

The following 15 questions were used:

1. What is your primary role in relation to ontology authoring?

2. How familiar are you with SUMO/SigmaKEE ontology authoring workflows?

3. Which of the following tools have you used for ontology authoring?

4. How often do you improve, refine, or extend existing terms in the SUMO ontology?

5. How often do you create brand-new terms in the ontology?

6. Which parts of the ontology authoring workflow do you perform most often?

7. Which part of the ontology authoring workflow do you find hardest or most confusing?

8. How confident do you feel when performing ontology authoring tasks in your current workflow?

9. Where do you encounter setup/configuration challenges?

10. How often do you switch between tools during the ontology authoring workflow?

11. At what point are you more likely to feel stuck or lose confidence?

12. What are the most common errors or problems you encounter?

13. How useful would an inline guided "new term / improve term" workflow be to you?

14. Which kinds of support would help you most?

15. If you could change one thing instantly about the workflow for creating and improving terms, what would it be?

Survey Findings

Responses from 7 participants are summarized below, segmented by user group. Participant identities are anonymized.

Participant Group A: Ontologists and Ontological Engineers (1 of 7 respondents, formal specification work, daily or weekly use)

This respondent uses SigmaKEE and command-line scripts as their primary tools. They are comfortable with KIF syntax and theorem provers but report that the workflow between editing and proving requires too many context switches. The primary pain point is the raw Vampire output: TPTP format is difficult to interpret without a translation layer. Setup friction, while painful initially, is a one-time cost for this user. Their top requested features were faster search, better error messages, and cleaner filtered proof output.

Participant Group B: Students and New Practitioners (5 of 7 respondents, new to ontology or early in training, technically capable)

This group represents the majority of respondents. They are learning ontology authoring as part of coursework or directed study, or are early-career practitioners entering the field. They do not know where to start in SigmaKEE. The Browse page presents thousands of terms with no guidance on entry points. They have attempted to write axioms but received Vampire errors they could not interpret. Setup friction is a significant barrier: several tried SUMOjEdit on macOS and gave up due to path configuration issues. The reuse-refine-create decision is the hardest step: 71.4% identified it as the most confusing part of the workflow. This group drives the majority of the quantitative findings and is the primary motivation for the guided discovery and decision panels in the wizard.

Participant Group C: Domain Experts (approximately 1 of 7 respondents, subject matter expert, variable technical background)

This respondent has domain knowledge to contribute but limited familiarity with KIF syntax or theorem provers. Their workflow is the most friction-heavy: they must simultaneously learn the tool, the ontology structure, and the formal language. Their top requested change was "existing term recommendation" and a workflow integrated into a single environment. This group represents the most ambitious design target: if the wizard works for them, it works for everyone.

Key quantitative findings across all groups:

- 71.4% identified "deciding whether to reuse, refine, or create a term" as the hardest or most confusing workflow step (driven primarily by Group B, which is 5 of 7 respondents)

- 85.7% said suggestions for similar existing terms would help them most

- 71.4% reported neutral or low confidence when performing ontology authoring tasks

- 100% of respondents found the inline guided workflow at least somewhat useful (71.4% somewhat useful, 14.3% very useful, 14.3% extremely useful)

- 57.1% cited initial installation and setup as a configuration challenge (primarily Group B)

- Top tools used: SigmaKEE (71.4%), command-line scripts (57.1%), SUMOjEdit (42.9%), VS Code plugin (28.6%)

3. User Analysis

Roles

Three roles are possible in our application:

- Ontologist / Ontology Engineer: Creates, edits, and validates axioms in SUMO/SUO-KIF. Runs proofs to check consistency. Represented by 1 of 7 survey respondents (Group A).

- Student / New Learner of Ontology: Learning ontology authoring through coursework or directed study. May also include early-career practitioners entering the field. Represented by 5 of 7 survey respondents (Group B), the majority of the survey population.

- Domain Expert: Possesses deep knowledge in a specific field (e.g., military operations, medicine, law) but has little or no experience with ontology tools, KIF syntax, or theorem provers. Needs to contribute domain knowledge without learning the full technical stack. Represented by 1 of 7 survey respondents (Group C).

We are building towards an interface that enables the tertiary user to download their domain expertise into the system with ease and skill, in an effort to create a world model knowledge representation for autonomous reasoning in AI applications.

User Characteristics Table

Primary User: Ontology Engineer

Characteristic	Description
Role	Creates, edits, and validates axioms in SUMO/SUO-KIF. Runs proofs. Maintains the knowledge base.
Task domain	Ontology authoring, axiom validation, theorem proving, KB maintenance
Frequency of use	Daily
Computer skill level	High. Comfortable with command line, code editors, and server configuration
Typing skills	Proficient. Regularly writes formal logic expressions

Editor experience	Extensive VS Code and SigmaKEE Browser usage
Formal training	Graduate-level CS with strong background in first-order logic and ontology engineering
Special considerations	Wants efficiency over guidance. Frustrated by too many clicks between editing and proving. Needs batch operations and session management.

Secondary User: Graduate Student

Characteristic	Description
Role	Searches and inspects existing terms and axioms. Learning to author axioms through coursework.
Task domain	Ontology exploration, learning KIF syntax, writing first axioms, understanding proof output
Frequency of use	Weekly during coursework or thesis work
Computer skill level	High in general CS (Python, Java) but new to ontology-specific tools
Typing skills	Proficient with code but not with KIF syntax
Editor experience	Familiar with VS Code for programming. No prior experience with SigmaKEE or SUMOjEdit
Formal training	CS graduate coursework in progress. Limited exposure to first-order logic
Special considerations	Does not know where to start. Needs guided onboarding, syntax examples, and plain-language explanations of proof output. Setup and configuration is a major barrier.

Tertiary User: Domain Expert (SME)

Characteristic	**Description**
Role	Contributes specialized domain knowledge (e.g., military operations, medicine, law) to the ontology. Does not write KIF directly.
Task domain	Describing concepts in natural language, validating whether formal representations match domain reality, reviewing axioms for correctness
Frequency of use	Occasionally, during knowledge elicitation sessions
Computer skill level	Moderate. Comfortable with standard office software and web browsers but unfamiliar with code editors or command line
Typing skills	Proficient with general text. No experience with formal logic syntax
Editor experience	No prior experience with VS Code, SigmaKEE, or similar tools
Formal training	Graduate-level or professional training in their own field. No formal CS or logic training
Special considerations	Needs a natural-language pathway to contribute knowledge without learning KIF. The interface must translate domain expertise into formal representations. Cannot interpret raw Vampire output. Relies on the ontology engineer to validate the formal encoding.

Persona 1 : Michael, the New Graduate Student

Attribute	Detail
Age	27
Role	Ontology browser, transitioning to ontology author
Background	First-quarter CS graduate student. Has taken courses in programming and databases but has no prior experience with formal ontologies or theorem provers. Enrolled in an ontology course this quarter.
Goals	Understand the SUMO hierarchy, learn KIF syntax, and write a first axiom that passes a consistency check.
Frustrations	Does not know where to start in SigmaKEE. The Browse page shows thousands of terms with no guidance. Tried to write an axiom but got a Vampire error that was impossible to understand. Tried SUMOjEdit on macOS but gave up; now uses VS Code.
Technical knowledge	Strong in Python and Java. No experience with first-order logic or ontology tools.
Frequency	Uses SigmaKEE 2-3 times per week for coursework.
Quote	"I know what I want to say, but I do not know how to say it in KIF."

Persona 2 : Dr. Corleone, the Experienced Ontologist

Attribute	Detail
Age	45
Role	Ontology author and system administrator
Background	Senior researcher who has been working with SUMO for over 15 years. Contributes axioms regularly and advises students. Comfortable with KIF syntax and Vampire output.
Goals	Author complex axioms efficiently, run batch proofs, and quickly identify inconsistencies. Wants to spend less time on repetitive navigation and more time on reasoning.
Frustrations	The interface requires too many clicks to go from editing to proving and back. No way to save a working session or bookmark frequently used terms. Proof output is raw text with no filtering.
Technical knowledge	Expert in first-order logic, SUMO, KIF, and TPTP. Comfortable with command line.
Frequency	Uses SigmaKEE daily.

Quote	"The tool works, but it fights me every step of the way."

Persona 3: Mr. Hagen the Domain Expert (SME)

Attribute	Detail
Age	52
Role	Domain expert contributing specialized knowledge. Does not write KIF directly.
Background	Attorney and certified public accountant with 25 years of experience in corporate law and financial regulation. Deep expertise in tax law, contract obligations, and regulatory compliance. Uses standard office software and legal databases (Westlaw, LexisNexis) daily but has never worked with ontology tools, formal logic, or theorem provers. Asked by a research team to help formalize legal and financial concepts in SUMO.
Goals	Contribute formal representations of legal and financial concepts to SUMO without needing to learn the full technical stack. Wants the system to preserve the nuance his domain requires.
Frustrations	Formal tools built for engineers do not accommodate the semantic precision that legal work demands. Ontology jargon does not map to how legal practitioners think about concepts. Cannot contribute without a collaborator who knows KIF.
Technical knowledge	Proficient with office software, legal databases, and spreadsheets. No programming, formal logic, or ontology experience.

Frequency	Occasionally, during knowledge elicitation sessions (1-2 times per month).
Quote	"In law, one word changes everything. I need to know your system respects that."

4. User Journey Map

Three journey maps are provided, one per user type. Each map is a to-be (desired state) map showing the experience the wizard is designed to provide. A current state (as-is) summary is included first to establish the baseline and make the design rationale explicit.

Current State Summary

Today there is no guided path for contributing to SUMO. The workflow is the same for all user types: find the GitHub repository, navigate to external documentation on an outside wiki, install the tool stack manually with 10 or more environment variables configured by hand, open a plain KIF file in a text editor, and attempt to validate and prove terms without scaffolding or feedback. The assumption that any user can complete this independently is the primary design problem this project addresses.

For User 1 (ontologist or engineer): The workflow exists but depends on tribal knowledge. Adding a new term means opening SigmaKEE in a browser, running a manual KB search, writing KIF from memory, running KIFFileChecker from the command line, manually editing config.xml, and invoking Vampire with the correct flags. Each tool is a separate window. There is no checklist, no validation loop, and no structured proof pipeline.

For User 2 (student or new practitioner): The workflow breaks at the first undocumented step. A student following the 1,300-line developer guide reaches Step 0 ("Write an inference a theorem prover should derive") with no explanation of what inference means. Installation requires 30 or more minutes. The gap between understanding ontology concepts and writing a valid provable KIF axiom has no bridge.

For User 3 (domain expert): There is no path in at all. CONTRIBUTING.md is written for Java developers. There is no mechanism for a domain expert to contribute knowledge without

first learning KIF, TPTP, and the full tool stack. The knowledge extraction problem is entirely unsolved by the current tooling.

Journey Map: User 3 (Domain Expert)

The journey map below traces a Domain Expert through the full ontology authoring experience using the proposed wizard. This persona has deep knowledge in a specific field but no prior experience with KIF or theorem provers. Five stages are mapped across five dimensions: what the user is doing, thinking, feeling, the pain points they encounter, and the design opportunities the wizard addresses at each stage.

	Stage 1: Setup and Onboarding	Stage 2: Scoping (I2L / LLM Iteration Chat)	Stage 3: Discovery and Decision	Stage 4: Authoring and Classification	Stage 5: Validate, Prove, and Submit
Doing	Cloning repo, running install script, configuring environment variables, starting the server, installing additional tools separately	Opening the wizard, describing the concept in natural language, answering agent edge-case questions, identifying what should be verifiable as a result of the concept	Reviewing in-interface search results, evaluating whether an existing term matches, choosing between reuse, improve, specialize, or create new	Writing the documentation string, selecting term type, finding the parent class, writing capability and purpose assertions, writing inference-enabling rules	Running KIFFileChecker, running consistency check, submitting a proof conjecture to Vampire, interpreting the result, routing the term to the correct file for review, exporting the proof and scenario file, deciding whether to contribute the proof example to the shared suggestion pool
Thinking	“How do I get this working? Why do I	“Is this concept already in SUMO? Am	“Is that existing term really what I mean? Is my	“What is the right parent class? Am I missing any	"Why did it time out? Is this inconsistent

	need to configure this manually?” Which repo has the tool I need ot talk about my knowledge?”	I describing it precisely enough? What would I want to be able to prove about this?”	concept a specializatio n of that, or something different? Do I actually need a new term?”	factual claims? Do my rules actually cover what I wrote in the documentati on?”	or just not provable yet? Who reviews this, and how long does it take?"
Feeling	Frustrated, overwhelme d	Curious, uncertain	Uncertain, investigating	Engaged but anxious	Tense, then relieved when proof is found
Pain Points	57.1% of respondents cited initial setup as a challenge. Environment variable configuratio n is manual. Tools are split across multiple repos with no unified install. The server must be started separately.	No structured way to capture proof goals during ideation. Easy to lose the thread of the original intent by the time authoring begins. Plain-langua ge descriptions do not map clearly to formal concepts without guidance.	71.4% find the reuse-refine-create decision the hardest step. KB search requires knowing the exact term name. No in-interface display of related terms during search. "Similar term" is ambiguous: is it a specializatio n, a sibling, or something adjacent?	Documentati on completenes s is invisible. No prompting to check whether the documentati on string maps to formal rules. Finding the right parent class requires deep familiarity with the SUMO hierarchy. No feedback on whether capability and purpose assertions are present.	Vampire output is raw TPTP format with no translation. Only three possible outcomes (proof found, counter-satis fiable, timeout) and no actionable guidance for the failure cases. Unclear where to submit and who reviews the contribution.
Opportunitie s	One-comma nd install via package manager. Setup wizard with progress indicator.	LLM chat panel that saves proof goals throughout the session. Plain-langua ge prompts:	In-interface search results displayed without leaving the editor. 85.7% of	Step-by-step panel sequence with one task per screen. Documentati on string analyzer that	Translated Vampire output in plain language. Routing back to the specific

	Automatic path and environment detection. Unified tool download that includes all dependencies.	"Describe a situation where you would need this concept, and what should be verifiable as a result." Agent pushes back with edge cases to sharpen the concept. Future direction: multimodal input panel ("Show us the way you see the world") allowing photo or voice as entry point. Sumo mascot spinner during KB load.	respondents want similar-term suggestions surfaced automatically. Three-way decision panel with examples for each branch (reuse, improve, specialize, create new). Confirmation checkpoint before advancing.	highlights claims without backing rules. Sibling suggestions prompted during parent-finding. Audio confirmation when each definition component is saved.	authoring step responsible for the failure. Submission routed automatically to the correct KIF domain file. Pease or domain expert review queue with draft status. Audio and visual confirmation when proof is found.

Quantitative Insights

Stage	**Key Data Point**	**Source**
Discovery and Decision	71.4% find reuse-refine-create decision hardest	Survey Q7
Discovery and Decision	85.7% want similar-term suggestions	Survey Q14
Authoring and Classification	71.4% report neutral or low confidence	Survey Q8

Validate, Prove, and Submit	100% found wizard concept at least somewhat useful	Survey Q13
Setup and Onboarding	57.1% report initial setup as a challenge	Survey Q9

Takeaways

The three highest-priority design interventions, ranked by survey support (note the small sample size of 7 people):

1. Similar-term suggestions surfaced during the Discovery phase (85.7% want this). This directly addresses the hardest reported step.

2. Guided reuse-refine-create decision support (71.4% find this hardest). A structured decision panel with examples reduces the cognitive load of the most failure-prone transition in the workflow.

3. Translated proof output and actionable failure routing (supports the 71.4% who lack confidence). Raw TPTP output is the final barrier between a user completing a contribution and giving up.

Journey Map: User 1 (Power User / Ontologist or Engineer)

	Stage 1: Direct Entry	Stage 2: Discovery	Stage 3: Authoring	Stage 4: Validate and Submit
Doing	Opening the wizard in direct-entry mode or dropping directly into KB search, skipping onboarding	Grepping .kif files, running KB Browse, evaluating exact and near matches, deciding on new term versus improvement to existing	Writing axioms from memory, running documentation completeness check, adding domain and range assertions	Running KIFFileChecker, invoking Vampire, reading TPTP output, routing term to correct file, exporting proof scenario file
Thinking	"I know what I want. Let me skip the wizard and get to work."	"Is there already a term for this? What's the closest parent? Am I creating a redundancy?"	"Am I missing any claims in the doc string? Are my rules tight enough to avoid spurious proofs?"	"Why is this timing out? Is it a missing axiom or a genuine unprovability? Who should review this?"

Feeling	Efficient, focused	Methodical	Confident but careful	Analytical, then satisfied
Pain Points	Current SigmaKEE requires browser window context switches between editing and proving. No session state between edits.	KB search requires knowing exact term names. No keyboard shortcut for jumping to related terms. Grep is external to the editor.	No inline documentation coverage feedback. Parenthesis errors are found late, at checker time, not during authoring.	Vampire output is unfiltered raw TPTP. No way to bookmark a proof or export the transcript without manual copy-paste.
Opportunities	Direct-entry mode bypasses all guided phases. Full keyboard shortcut support. Session persistence with bookmarks for frequently used terms.	Fuzzy search with automatic similar-term surfacing inside the editor. One-click jump to parent, sibling, or child terms.	Real-time parenthesis balance indicator. Inline documentation string analyzer. Batch axiom paste from clipboard.	Filtered Vampire output with relevant clauses highlighted. One-click proof export as scenario file. Opt-in contribution of proof transcript to shared example pool (off by default).

Journey Map: User 2 (Student / New Practitioner)

	Onboarding	**Exploring the KB**	**Scoping (LLM Chat)**	**Authoring (Guided)**	**Validate and Submit**
Doing	Installing the wizard extension, completing the setup checklist, loading a starter KB, running the example proof to confirm Vampire	Browsing existing SUMO terms, reading documentation strings, following parent-child links, searching for a concept they already	Opening the scoping chat, describing their concept, receiving edge-case pushback, forming a plain-language proof goal	Following the step-by-step panel sequence, writing their first documentation string, getting feedback on uncovered claims,	Running KIFFileChecker, reviewing the plain-language result summary, submitting a conjecture, reading the translated proof output

	works	know		writing their first rule	
Thinking	"Is this installed correctly? How do I know if Vampire is working?"	"How does SUMO organize things? What level of specificity should my term be at?"	"Is my concept already captured somewhere? Am I describing something genuinely new?"	"What is the right parent class? Is my rule actually saying what I think it says?"	"What does counter-satisfiable mean? Did I write something contradictory?"
Feeling	Uncertain, checking	Curious, exploring	Engaged, uncertain	Focused, cautious	Anxious, then relieved
Pain Points	No "hello world" proof example exists today. Students cannot verify a working install without already knowing how to write a conjecture.	SUMO hierarchy is 25,000+ terms with no guided entry point. Parent class selection requires deep prior knowledge.	Proof goals are an abstract concept for someone who has not used a theorem prover before.	Documentation completeness check is invisible in the current tools. Vampire errors are cryptic.	Raw TPTP output is uninterpretable without prior exposure. No explanation of what the three result types mean.
Opportunities	Bundled example proof runs automatically after install to confirm the environment works. Green check and mascot animation on success.	Interactive hierarchy browser with plain-language summaries at each level. Hovering a SUMO term shows its definition and example usage.	Plain-language prompt: "Describe a situation where you would need this concept, and what should be verifiable as a result." No mention of "inference" until the user has seen an example.	Progressive disclosure: start with documentation string and parent class only; reveal domain, range, and rule panels after first two are complete. Inline worked examples for each axiom type.	Plain-language translation of all three Vampire result types. "What went wrong?" panel that explains counter-satisfiable and timeout in terms of the axioms the user just wrote.

5. Task Analysis

Key Efficiency Metric: Click Count

A primary UX metric for this redesign is the number of discrete user actions required to complete a term from concept to validated submission. This metric, sometimes called interaction count or taps to complete, is a foundational measure in interaction design: reducing unnecessary clicks directly reduces cognitive load, error rate, and time-to-completion. Apple's Human Interface Guidelines identify minimizing the number of steps required to complete a task as a core design principle (Apple Inc., Human Interface Guidelines, developer.apple.com/design/human-interface-guidelines). The principle is validated in everyday experience: non-technical users across all age groups navigate iPad applications fluently because the interface reduces each task to its minimum required steps. We adopt click minimization as a measurable target for the same reason: if the goal is to lower the barrier to entry for SUMO contribution, the interaction cost of each step is not incidental, it is the barrier.

Current baseline (existing SigmaKEE workflow): approximately 20 to 30 discrete user actions across 3 to 5 separate tools to take a concept from initial idea to a validated, submitted term. This includes manual environment setup steps, context switches between the browser, editor, and prover, and manual interpretation of raw Vampire output.

Wizard target: reduce to a single linear panel sequence in VS Code where each stage requires at most 1 to 3 user decisions. The goal is to reach a validated, submitted term in under 15 total interactions for a domain expert with no prior KIF experience.

Measurement plan: count clicks per task in the HTA below for the current workflow. Recount for the wizard prototype during usability evaluation. Track reduction as the primary efficiency metric alongside task-completion time, error rate, and user-reported confidence.

Scenario Design

The scenario centers on a sports domain expert who has been asked to contribute to the SUMO knowledge base. They want to formalize the concept of the Super Bowl as a recurring major American sporting event. They have no prior experience with SUO-KIF or theorem provers. They open the wizard in VS Code and begin.

This scenario was selected because it exercises every branch of the pipeline: the user finds a near-match term (Bowl) that is related but fundamentally different in nature, must justify a new term, must walk down the SUMO taxonomy from a general category toward a specific event type, must write formal rules for a concept they understand deeply but have never expressed in logic, and discovers mid-process that a second related term (SuperBowlTrophy) is also needed. The scenario mirrors what Onur and Jonathan worked through in the ideation session as a test of the full decision tree.

User: A sports historian and journalist. Deep knowledge of American football history, rules, and culture. No formal logic background. Uses VS Code daily for writing.

Goal: Add a formal SUMO term for the Super Bowl game event so that autonomous reasoning systems can make accurate inferences about American sporting culture and championship outcomes.

01. Scope the Concept
    a. Open the wizard panel in VS Code / Rust CLI / application window UX / sigmaKEE browser / SUMOjEdit
    b. Read the entry prompt: “Tell us about the concept you want to contribute. Describe it in your own words or drop an image / upload.”
    c. Type: "The Super Bowl is the annual championship game of the NFL. Two conference champions compete. The winner is considered the NFL champion for that year."
    d. Agent responds: "Great. Can you describe a situation where knowing something is a Super Bowl game would matter? What should be verifiable as a result?" (plain-language translation of an inference goal prompt)
    e. User responds: "If a team wins the Super Bowl, they should be recognized as the NFL champion for that season."
    f. Agent saves proof goal: winner of SuperBowlGame implies NFLChampion status for that season (ex. edge case to iterate with the agent on: Does the Champion status spans until a new winner is crowned at the following years super bowl - which means there's a reigning super bowl champion until that year but also a singular year XXXX superbowlChampion)
    g. Agent asks: "Is the Super Bowl one specific game, or a type of event that happens every year?" User selects: a type of event that recurs annually
    h. Exit condition: concept description and proof goal saved; proceed to Discovery
02. Discovery (NOTE - THIS WIZARD SHOULD ALSO BE IMPLEMENTED AND OUGHT TO BE WITH A DETERMINISTIC BACKEND WITH A TEMPLATE RESPONSE TO CONSTRAIN LLM GUARDRAILS TO ROUTE TO DECISION TREE WORKFLOW CHART)
    a. Wizard runs automated search for "SuperBowl," "Super Bowl," and "Bowl" in the loaded knowledge base
    b. Results returned: Bowl exists as a subclass of Container (a physical object you put things in)
    c. Agent presents result: "We found Bowl. Is your Super Bowl a kind of container, or a different kind of thing?" with examples
    d. User selects: "A different kind of thing. It is an event."
    e. Wizard surfaces related event terms: Game, Contest, SportingEvent
    f. Agent asks: "Does the Super Bowl fit under any of these, or is it something more specific?" User confirms: it is a type of sporting contest

g. Agent asks: "Is it a specialization of one of these terms, or does it need its own name because it is specific to American football?" User confirms: needs its own name
h. Agent confirms routing: "We will create a new term. Your concept is not a container. It is a specific recurring sporting event." Confirmation checkpoint.\\
i. Proceed to Classification

03. Classification
a. Agent asks: "Is the Super Bowl a type of thing (a class), or one specific thing (an instance)?" User selects: a type of thing that has many instances (e.g., Super Bowl 2024, Super Bowl 2025)
b. Wizard routes to CLASS
c. Agent asks: "Does the Super Bowl exist as a physical object, or does it happen over a period of time?" User selects: it happens over time
d. Wizard routes to Process in the SUMO hierarchy
e. Agent walks taxonomy: Process, SocialInteraction, Contest. Agent asks: "Is there a more specific category for competitive sports events?" User confirms: yes, it is a sporting contest
f. Agent asks: "Can you think of similar events at the same level? For example, other annual championship games?" User names NBA Finals, World Series, Stanley Cup Finals
g. Agent surfaces a second term need: "You mentioned the Super Bowl Trophy earlier. That is a physical object, not the game event itself. You may need a separate term for the trophy. We will note that and you can return to it." SuperBowlTrophy flagged as a follow-up term
h. Parent confirmed: AmericanFootballGame (or Contest, depending on what exists in the loaded KB)
i. Confirmation checkpoint. Proceed to Implementation

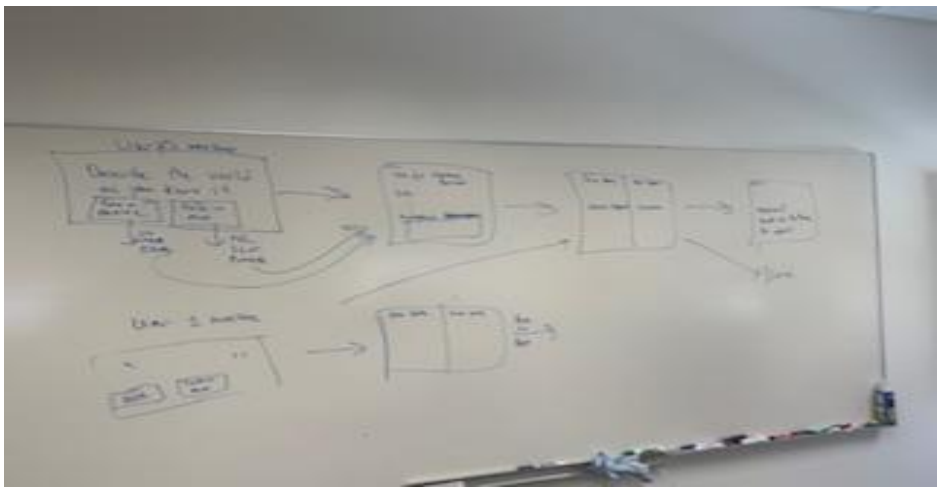
04. İmplementation
a. Wizard opens definition panel. Prompt: "Write a subclass assertion placing SuperBowlGame under its parent."
b. Prompt: "Write a documentation string. What should someone know if they encounter this term?"
c. User writes documentation string. Agent parses it and highlights phrases that have no backing rule yet.
d. Agent prompts for each uncovered claim: "You wrote that the winner is recognized as the champion. Can we write a rule that captures that?" User writes the hasPurpose or inference rule.
e. Agent prompts for capability assertion: "What does a SuperBowlGame afford to the participants?" User writes capability assertion.
f. At least one inference-enabling rule is confirmed present. Documentation string coverage check passes.
g. Wizard automatically checks WordNet mapping for "Super Bowl." Result displayed.

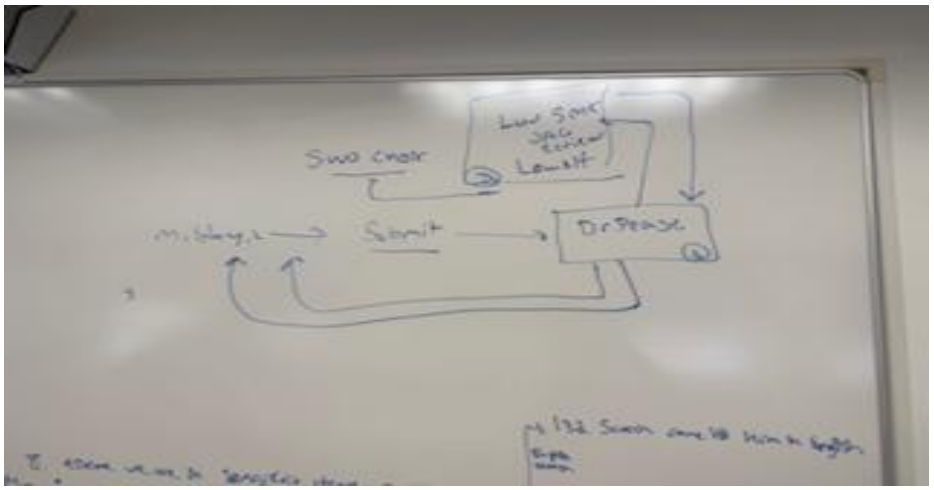
h. KIFFileChecker runs. Parenthesis balance and syntax confirmed. Sumo mascot spinner shown during check.
i. Consistency check runs using the fast Rust-based backend. Result displayed with plain-language summary.
j. User submits proof conjecture: if a team is the winner of a SuperBowlGame, they hold the status of NFLChampion for that season.
k. Vampire runs. Wizard displays result in plain language: "Proof found" routes forward. Counter-satisfiable or timeout routes back to the specific authoring step that likely caused the failure, with an explanation.
l. Readiness checklist confirmed: documentation string present, parent class verified, domain and range for all predicates, at least one inference-enabling rule, no variable scoping issues, prover returned Theorem, WordNet mapping in place, KIFFileChecker passed.
m. Audio confirmation plays.

05. Submit and Review
    a. Wizard asks: "Which domain file should this go into?" Options presented based on loaded KB. User selects sports.kif or development.
    b. Term is saved with draft status and routed to the development directory.
    c. Notification sent to assigned ontology reviewer (Pease or domain expert group).
    d. User receives feedback asynchronously. Wizard reopens at the relevant step for revision.
    e. After reviewer approval, term is merged into the main SUMO knowledge base.
    f. Follow-up flagged: SuperBowlTrophy term still needs to be authored. Wizard offers to begin a new session.

6. Conceptual Design

The below are some conceptual interfaces during ideation:





Objects (Nouns)

These are the primary entities that users think about and interact with:

Object	Description
Term	A concept in the SUMO ontology (e.g., HostileAct, CognitiveAgent, Nation), meaning some entity we understand about the world
Axiom	A formal statement in KIF that defines a rule or relationship between terms, which is either True or False
Knowledge Base	A collection of terms and axioms loaded into the system (e.g., Merge.kif)
Proof	The output produced by the theorem prover after evaluating a conjecture.

Conjecture	A statement submitted to the theorem prover to be proved or disproved, which the ATP negates and then attempts to prove false
Reasoning Step	A single inference step within a proof
Search Result	A list of terms returned after a search query
Session	A working period including terms browsed, axioms authored, and proofs run

Attributes (Adjectives)

Object	Attributes
Term	Name, definition, hierarchy depth, number of subclasses, number of axioms, documentation string
Axiom	KIF expression, syntax status (valid/invalid), author, date created, associated terms
Knowledge Base	Name, file path, number of terms, number of axioms, connection status, load time
Proof	Result status (proved/disproved/timeout), duration, number of reasoning steps, prover used
Conjecture	KIF expression, syntax status, target terms
Reasoning Step	Step number, inference rule used, source axioms, resulting clause
Search Result	Query string, number of matches, relevance ranking
Session	Start time, duration, number of actions performed

Relationships between Objects

- A Knowledge Base contains many Terms.
- A Knowledge Base contains many Axioms.
- A Term has zero or more parent Terms (superclasses).
- A Term has zero or more child Terms (subclasses).
- A Term has zero or more sibling Terms (same parent).
- An Axiom references one or more Terms.
- A Conjecture is a special type of Axiom submitted for proving.
- A Proof is generated from one Conjecture.
- A Proof contains one or more Reasoning Steps.
- A Reasoning Step references one or more Axioms from the KB.
- A Search Result contains zero or more Terms.

- A Session contains zero or more Axioms, Proofs, and Search Results.

Actions

Actions on Objects

Action	Object	Description
Search	Term	Find a term by name or partial match
Browse	Term	Navigate the ontology hierarchy
Create	Axiom	Write a new axiom in the editor
Delete	Axiom	Remove an axiom from the working session
Duplicate	Axiom	Copy an existing axiom as a starting point
Submit	Conjecture	Send a conjecture to the theorem prover
Export	Proof	Save proof results to a file
Connect	Knowledge Base	Load and activate a KB
Disconnect	Knowledge Base	Unload a KB
Start	Session	Begin a new working session
Resume	Session	Return to a previously saved session

Actions on Attributes

Action	Attribute	Description
Edit	Axiom expression	Modify the KIF text of an axiom
Validate	Axiom syntax status	Check if the KIF expression is syntactically correct
Rename	Session name	Change the label of a saved session
Expand/Collapse	Reasoning step	Show or hide proof step details
Filter	Search result ranking	Narrow results by type, relevance, or hierarchy position

Actions on Relationships

Action	Relationship	Description
Navigate up	Term to parent Term	Move from a term to its superclass
Navigate down	Term to child Term	Move from a term to its subclass
Link	Axiom to Term	Associate an axiom with a specific term
Trace	Reasoning Step to Axiom	Follow a proof step back to its source axiom

Switch	Session to Knowledge Base	Change which KB is active